Diapositiva 06:
Versiones de SQL Server

El Origen (1989)

- **Sybase / Microsoft / Ashton-Tate:** SQL Server no nació como un producto 100% de Microsoft. Fue el resultado de una alianza para portar el motor de base de datos de Sybase al sistema operativo OS/2. Ashton-Tate (creadores de dBase) participó en el lanzamiento inicial, aunque se retiró pronto. Más adelante, Microsoft y Sybase separaron sus caminos y Microsoft reescribió el motor para Windows NT (dando lugar a SQL Server 7.0 y posteriores).

SQL Server 2005 (El inicio de la era moderna)

- **DMVs (Dynamic Management Views):** Vistas y funciones integradas que exponen el estado interno del servidor. Son la herramienta principal de un DBA para monitorizar el rendimiento, diagnosticar cuellos de botella (bloqueos, uso de CPU/memoria) y revisar el estado general de la instancia en tiempo real.

Ejecución y Conexiones (Monitorización en tiempo real)

- `sys.dm_exec_requests`: Muestra qué comandos se están ejecutando en el servidor en este instante exacto. Es vital para identificar consultas colgadas, ver qué sesión está bloqueando a otras (`blocking_session_id`) o comprobar el progreso porcentual (`percent_complete`) de operaciones largas como copias de seguridad o reconstrucción de índices.
- `sys.dm_exec_sessions`: Lista todas las conexiones autenticadas en el motor, revelando quién está conectado, el nombre de la máquina origen y desde qué programa o aplicación se conectan (`program_name`).
- `sys.dm_exec_sql_text`: Es una función (DMF) que se cruza con las anteriores pasando el `sql_handle`. Se encarga de traducir el identificador binario de la consulta a la sentencia T-SQL en texto plano que el usuario lanzó realmente.

Rendimiento e Índices (Optimización)

- `sys.dm_db_index_usage_stats`: Revela cuántas veces se ha usado un índice para leer datos (*seeks/scans*) frente a cuántas veces el motor ha tenido que gastar recursos en actualizarlo (*updates*). Es la herramienta principal para detectar y eliminar índices inútiles que ralentizan las escrituras sin aportar beneficio a las lecturas.
- `sys.dm_db_missing_index_details`: Contiene las recomendaciones del propio optimizador de SQL Server. Registra las columnas sobre las que el motor cree que faltan índices basándose en las consultas recientes, mostrando incluso la mejora de rendimiento porcentual estimada si se crearan.
- `sys.dm_exec_query_stats`: Almacena las estadísticas acumuladas de todas las consultas que están en la memoria caché. Permite a los DBAs extraer fácilmente el "Top 10" de las consultas que más CPU o memoria han consumido históricamente en el servidor.

Salud del Sistema (Detección de cuellos de botella)

- `sys.dm_os_wait_stats`: Es el punto de partida de cualquier diagnóstico de rendimiento avanzado. Registra los tiempos de espera del motor

agrupados por categoría. Si SQL Server va lento, esta DMV te dice exactamente el motivo físico: si espera por lectura de discos (`PAGEIOLATCH_SH`), por bloqueos entre usuarios (`LCK_M_...`) o por red (`ASYNC_NETWORK_IO`).

- `sys.dm_os_performance_counters`: Expone los mismos contadores de rendimiento de Windows (Performance Monitor) pero directamente consultables mediante T-SQL, permitiendo ver el uso de RAM, páginas por segundo o transacciones por segundo sin salir del entorno de base de datos.

Ejemplo:

```
SELECT
	r.session_id,
	s.login_name,
	s.host_name,
	r.status,
	r.command,
	t.text AS consulta_sql,
	r.blocking_session_id AS bloqueada_por,
	r.wait_type AS tipo_espera,
	r.wait_time / 1000.0 AS segundos_esperando,
	r.cpu_time AS cpu_ms,
	r.logical_reads AS lecturas_logicas,
	r.percent_complete AS porcentaje_completado
FROM
	sys.dm_exec_requests r
INNER JOIN
	sys.dm_exec_sessions s ON r.session_id = s.session_id
CROSS APPLY
	sys.dm_exec_sql_text(r.sql_handle) t
WHERE
	r.session_id <> @@SPID -- Excluye la sesión desde la que lanzas este script
	AND s.is_user_process = 1; -- Filtra los procesos internos del sistema (solo muestra conexiones
reales)
```

Se utiliza CROSS APPLY sys.dm_exec_sql_text(r.sql_handle) t porque la tabla dm_exec_requests no contiene la sql lanzada en texto sino un hexadecimal que es un handler a una funcion con la SQL. Debemos de traducir cada uno de estos handler si existen a su SQL textual y CROSS APPLY es un bucle que va linea a linea traduciendo esos handler en SQL textuales (amigables)

- **TRY/CATCH:** Introducción del manejo estructurado de excepciones directamente en Transact-SQL (T-SQL). Permitió a los desarrolladores capturar y gestionar errores de forma limpia, similar a lenguajes como C# o Java, superando la antigua necesidad de comprobar la variable @@ERROR tras cada instrucción.

Un ejemplo:

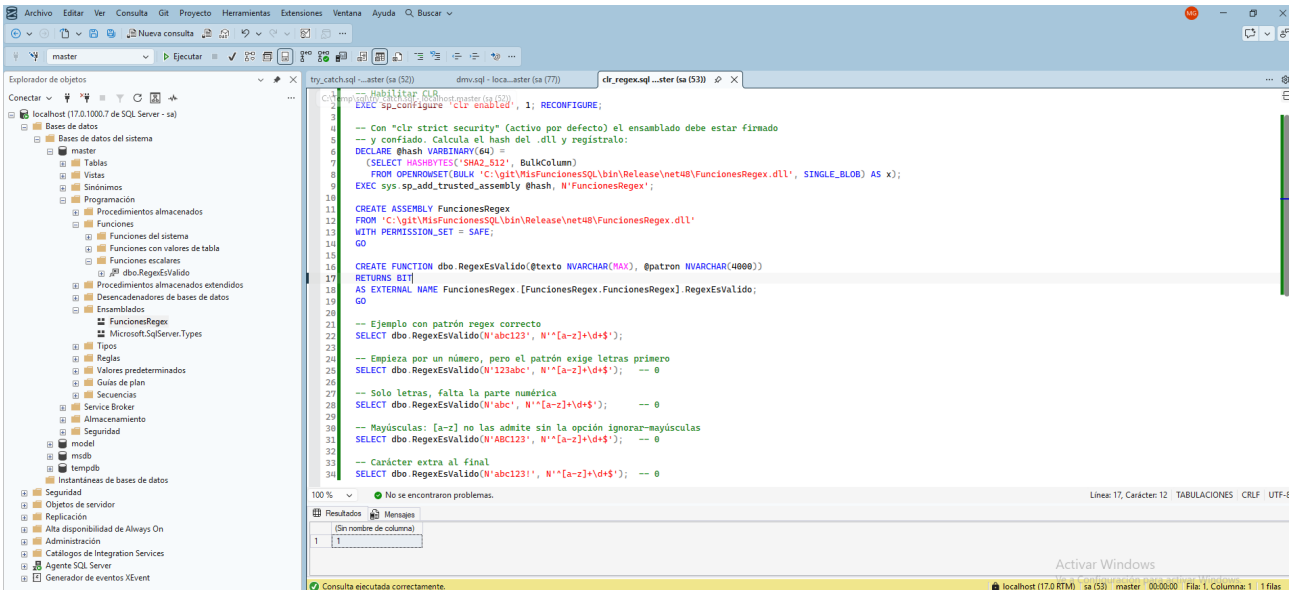
```
BEGIN TRY
	-- Intentamos una operación ilegal
	DECLARE @Resultado INT;
	SET @Resultado = 100 / 0;

	-- Si la línea anterior falla, el código salta, por lo que esto nunca se
ejecuta
	PRINT 'El cálculo fue un éxito.';
END TRY
BEGIN CATCH
	-- El control salta directamente aquí al producirse el error
	SELECT
		-- Código interno del error (ej. 8134)
		ERROR_NUMBER() AS NumeroError,
		-- Línea exacta donde ocurrió
		ERROR_LINE() AS LineaError,
		-- Texto descriptivo ('Divide by zero error encountered.')
		ERROR_MESSAGE() AS MensajeError;

END CATCH;
```

- **CLR (Common Language Runtime):** La integración del framework .NET dentro del propio motor de base de datos. Permite escribir procedimientos almacenados, desencadenadores (triggers) y funciones utilizando lenguajes como C# o VB.NET, ideal para operaciones matemáticas complejas, manipulación de cadenas o acceso a recursos externos donde T-SQL es ineficiente.

  Ver la implementación dll en git: C:\git\MisFuncionesSQL y C:\Temp\sql\clr_regex.sql para desplegarlo. Resultado y tests desde SQL Server Management Studio:

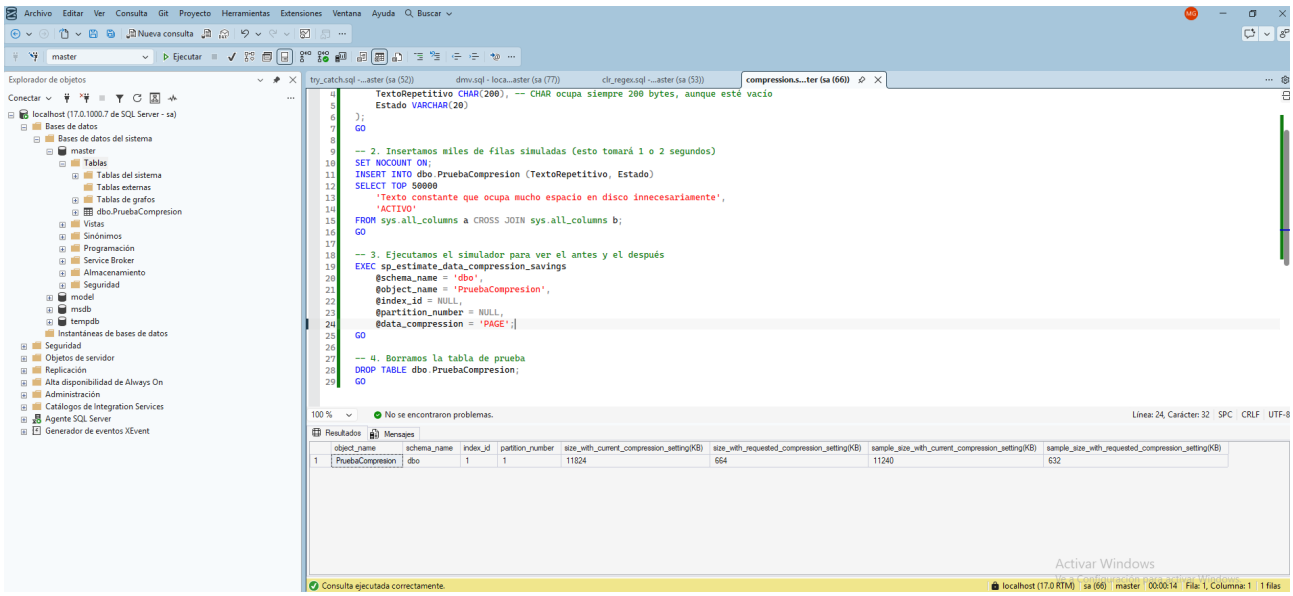


SQL Server 2008

- **Compresión:** Funcionalidad de compresión de datos a nivel de fila y de página. Permite reducir drásticamente el espacio de almacenamiento en disco y, lo que es más importante, reduce las operaciones de entrada/salida (E/S) al leer más datos en la misma cantidad de memoria RAM, a cambio de un ligero aumento en el uso de CPU.

  Ver un ejemplo simulado en y C:\Temp\sql\compression. Con estos resultados:

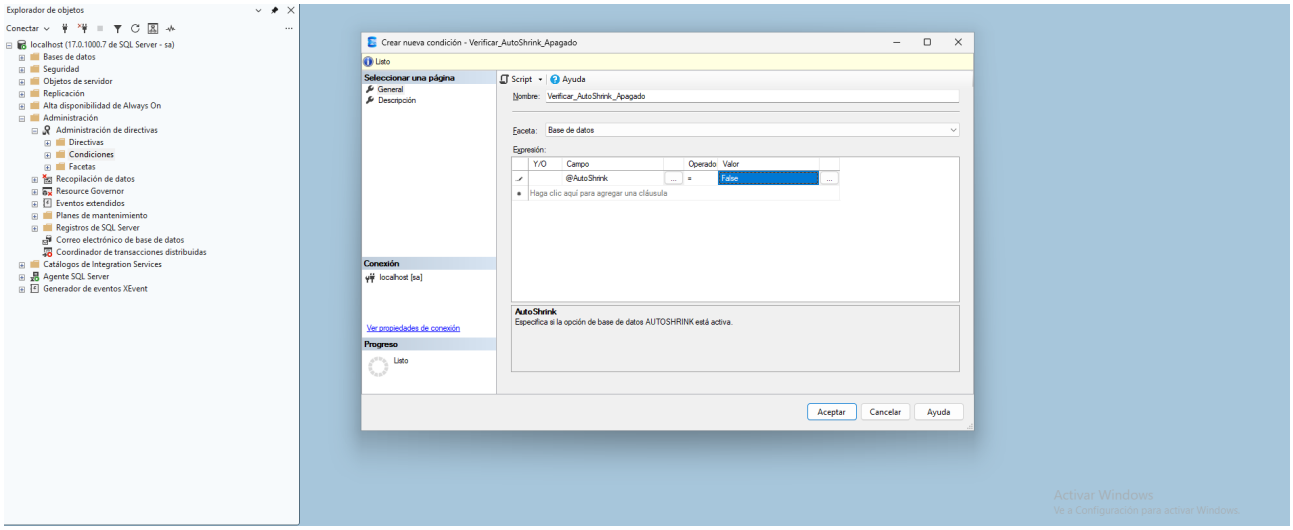
  - **object_name:** El nombre de la tabla evaluada, indicando que es "PruebaCompresion".
  - **schema_name:** El esquema al que pertenece la tabla dentro de la base de datos, en este caso "dbo".
  - **index_id:** El identificador del índice físico evaluado. El valor 1 significa que se está evaluando el índice agrupado (Clustered Index), es decir, la estructura principal donde residen los datos de la tabla ordenados por su clave primaria. (El valor 0 indicaría una tabla sin índice, conocida como *Heap*).
  - **partition_number:** El número de partición de la tabla. Como la tabla no está dividida en múltiples particiones de almacenamiento, todo reside en la partición predeterminada 1.
  - **size_with_current_compression_setting(KB):** El tamaño total que la tabla ocupa físicamente en el disco en este momento con su configuración original (sin compresión), alcanzando los 11824 KB.
  - **size_with_requested_compression_setting(KB):** La estimación del tamaño final de la tabla si aplicas la compresión, proyectando una caída drástica a solo 664 KB. Esto demuestra gráficamente la reducción de las operaciones de Entrada/Salida (E/S) mencionada en la teoría.
  - **sample_size_with_current_compression_setting(KB):** El tamaño de la muestra de datos reales (11240 KB) que el motor de SQL Server copió temporalmente para realizar la prueba matemática. Para no saturar la CPU en tablas de terabytes, el procedimiento toma una muestra representativa en lugar de evaluar cada registro.
  - **sample_size_with_requested_compression_setting(KB):** El tamaño resultante de esa muestra específica tras aplicarle el algoritmo de compresión en memoria durante la prueba, reduciéndose a 632 KB. SQL Server utiliza la diferencia entre estas dos últimas columnas de muestra para calcular la proyección total



- **Policy-Based Management (Administración basada en directivas):** Un sistema que permite a los DBAs definir, aplicar y monitorizar reglas de configuración en una o varias instancias de SQL Server. Por ejemplo, se puede forzar una directiva que exija que todas las tablas tengan una clave primaria o que la opción "Auto-Shrink" esté siempre desactivada.

  Ejemplos

- Como evitar activar Auto-Shrink a la hora de crear o actualizar un tabla. Creamos primero la condición: Verificar_AutoShrink_Apagado.



Después creamos la política Forzar_AutoShrink_Apagado y asociamos esa condición la aplicamos a toda la base de datos, añadimos como modo de evaluación **On change: prevent,** que creará un trigger DDL de servidor a nivel base de datos que previene que actimos este modo para cada una de las tablas que vayamos a crear o intentemos actualizar con este argumento autoShrink, disparando un error en caso de que queramos activarlo incumpliendo la policy recién creada.

- Como **no existe un PBM para poder chequera si una tabla se crea o no con cable primaria**, deberemos de utilizar un DDL Trigger a nivel de database sobre la creación de una tabla de este tipo:

```
CREATE TRIGGER trg_Forzar_PK_En_Tablas
ON DATABASE
FOR CREATE_TABLE
AS
BEGIN
    SET NOCOUNT ON;

    -- Extraer el esquema y nombre de la tabla desde el XML del evento DDL
    DECLARE @EventData XML = EVENTDATA();
    DECLARE @SchemaName SYSNAME = @EventData.value('(/EVENT_INSTANCE/SchemaName)
[1]', 'SYSNAME');
    DECLARE @TableName SYSNAME = @EventData.value('(/EVENT_INSTANCE/ObjectName)
[1]', 'SYSNAME');
    DECLARE @ObjectId INT = OBJECT_ID(QUOTENAME(@SchemaName) + '.' +
QUOTENAME(@TableName));

    -- Verificar si existe un constraint de tipo 'PK' para este nuevo Object ID
    IF NOT EXISTS (SELECT 1 FROM sys.key_constraints WHERE type = 'PK' AND
parent_object_id = @ObjectId)
    BEGIN
        PRINT 'ERROR: Política de Arquitectura. Toda tabla debe crearse
definiendo una Clave Primaria (Primary Key) inline.';
        ROLLBACK TRANSACTION;
    END
```

```
END;
```

SQL Server 2012

- **Always On AG (Availability Groups):** La evolución definitiva para Alta Disponibilidad (HA) y Recuperación ante Desastres (DR), reemplazando al antiguo "Database Mirroring". Permite agrupar varias bases de datos para que conmuten por error (failover) de forma conjunta, y ofrece la posibilidad de tener réplicas secundarias de solo lectura para descargar el tráfico de reportes del servidor principal.

  Database Mirroring fue la principal solución de Alta Disponibilidad (HA) y Recuperación ante Desastres (DR) introducida en SQL Server 2005, operando exclusivamente a nivel de base de datos individual antes de ser reemplazada por los Always On Availability Groups en 2012:

  - **Funcionamiento central:** Mantenía una única copia exacta (servidor espejo) de una base de datos de producción (servidor principal) enviando de forma continua los bloques del registro de transacciones (*transaction log*) a través de la red.
  - **Conmutación automática:** Soportaba una arquitectura de tres nodos incluyendo un servidor "Testigo" (*Witness*). Este actuaba como árbitro; si el servidor principal caía, el testigo confirmaba la caída y orquestaba un *failover* automático para que el espejo asumiera el rol principal en segundos.
  - **Limitación crítica (Copia ciega):** A diferencia de las réplicas modernas de Always On, la base de datos en el servidor espejo estaba permanentemente en estado `Restoring`. **No se podía leer ni consultar en tiempo real**, impidiendo usar esa infraestructura secundaria para balancear cargas de lectura o reportes (la única vía era crear "fotos" manuales o *Database Snapshots* temporales).
  - **Restricción 1:1:** Solo permitía un único destino. No era posible tener un espejo síncrono local para alta disponibilidad y otro asíncrono remoto para desastres de forma simultánea.
  - **No agrupable:** La configuración se hacía base de datos por base de datos. Si una aplicación dependía de cinco bases de datos, no garantizaba que las cinco conmutaran juntas al mismo tiempo.

- **Columnstore (Índices columnares):** Un cambio de paradigma en el almacenamiento. En lugar de guardar los datos por filas, los almacena por columnas y los comprime masivamente. Está diseñado específicamente para cargas de trabajo analíticas (OLAP / Data Warehousing), acelerando las consultas de agregación (GROUP BY, SUM, COUNT) de forma exponencial en tablas con millones o miles de millones de registros.

El DBA debe decidir explícitamente cuándo implementarlos, existiendo dos formas principales de hacerlo:

- **Agrupado (Clustered Columnstore Index - CCI):** Convierte la tabla entera al formato columnar, reemplazando por completo el almacenamiento tradicional por filas. Se utiliza en tablas de hechos masivas (Data Warehouses) donde las operaciones son principalmente cargas de datos en lote y lecturas de millones de registros, ya que este formato penaliza las actualizaciones y borrados puntuales frecuentes (`UPDATE` / `DELETE`).
- **No Agrupado (Nonclustered Columnstore Index - NCCI):** Se crea como un índice secundario sobre una tabla tradicional (*Rowstore*) que ya existe. Esto permite un modelo híbrido (conocido como HTAP), donde el sistema transaccional del día a día sigue usando las filas estándar para escrituras rápidas, mientras que los informes analíticos utilizan el índice columnar paralelo para hacer sumas y promedios a velocidad extrema.

En resumen, requieren un análisis previo de la carga de trabajo y un comando explícito (`CREATE CLUSTERED COLUMNSTORE INDEX...` o `CREATE NONCLUSTERED COLUMNSTORE INDEX...`) sobre las tablas específicas que sufran cuellos de botella en consultas de agregación.

El uso de índices *columnstore* implica un intercambio técnico estricto: sacrificas rendimiento en escrituras y consumo de recursos a cambio de velocidades extremas en lectura analítica.

Las penalizaciones y costes operativos se dividen en cuatro áreas críticas:

- **Degradación en consultas de búsqueda puntual (Seeks):** Si la aplicación necesita recuperar una fila específica (ej. `SELECT * FROM Clientes WHERE ID = 5`), el columnstore es significativamente más lento que un índice B-Tree tradicional. En lugar de navegar por un árbol para ir directo al dato, el motor debe ubicar el *Rowgroup* (grupo de filas), pasarlo a RAM y descomprimir cientos de miles de valores de esa columna para devolver un solo registro.
- **Penalización extrema en actualizaciones (UPDATEs):** Los segmentos de datos comprimidos en un columnstore son inmutables. Un `UPDATE` no modifica el dato en su sitio; se ejecuta físicamente como un `DELETE` lógico (marcando el registro original en un mapa de bits oculto, el *deleted bitmap*) seguido de un `INSERT` de la nueva fila en una estructura temporal sin comprimir. Esto fragmenta el índice y consume mucha CPU.
- **Alta demanda de memoria RAM (Memory Grants):** Construir el índice, así como descomprimir segmentos al vuelo durante las consultas, requiere asignaciones masivas de memoria. Si el servidor no tiene

suficiente RAM disponible, SQL Server expulsará la operación hacia la base de datos `tempdb` (en disco), lo que destruye instantáneamente el beneficio de rendimiento de la consulta.

- **La carga oculta del "Tuple Mover":** Para no penalizar los `INSERT`s individuales, los datos nuevos entran primero en el *Delta Store* (una estructura oculta en formato de fila tradicional). Cuando ese almacén alcanza exactamente 1.048.576 registros, un proceso en segundo plano llamado *Tuple Mover* despierta para comprimir esos datos en un nuevo segmento columnar. Este proceso de compresión intensiva genera picos severos de uso de CPU.

Conclusión arquitectónica: Si aplicas un índice columnstore sobre una tabla transaccional (OLTP) pura, con alta concurrencia de `UPDATE`, `DELETE` e `INSERT` de una en una fila, paralizarás el rendimiento del servidor. Su hábitat natural son tablas donde los datos se insertan en grandes bloques (cargas *batch*) y rara vez se modifican posteriormente.

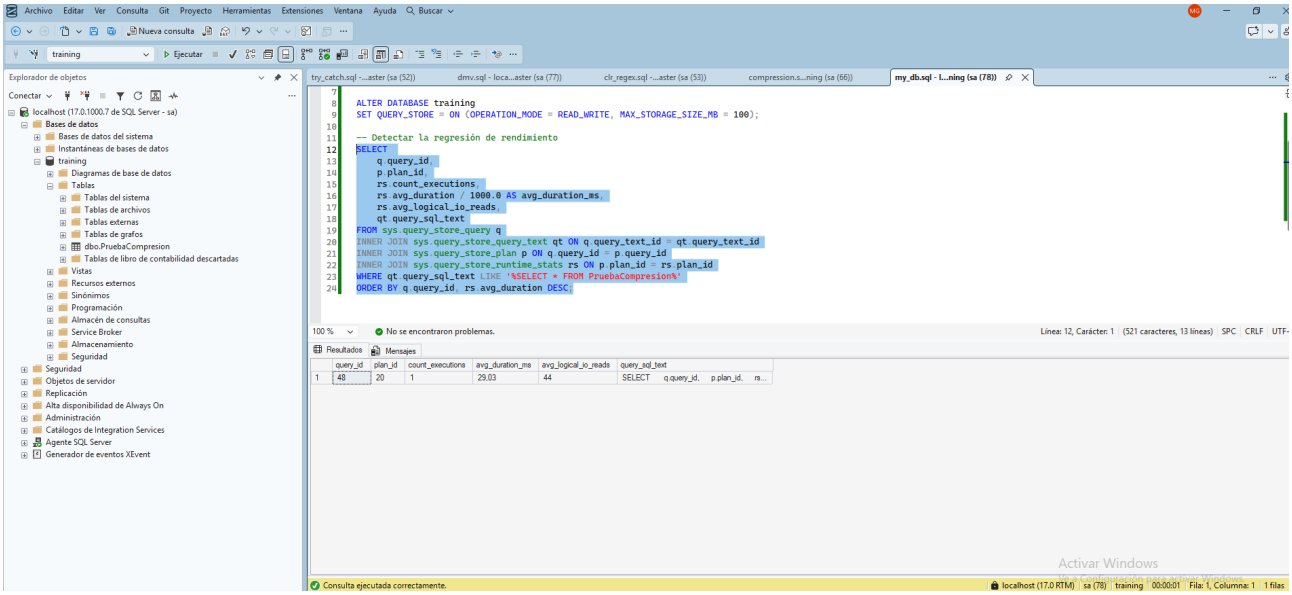
SQL Server 2014 (Rendimiento y cachés)

- **In-Memory OLTP (Hekaton):** Un motor de base de datos integrado y optimizado para la memoria principal. A diferencia de las tablas tradicionales, no utiliza bloqueos ni pestillos (*locks/latches*), sino control de concurrencia optimista multiversión. Permite crear tablas que residen en memoria y compilar procedimientos almacenados de forma nativa a código C, logrando un rendimiento extremo para sistemas transaccionales (OLTP) de alta concurrencia.
- **Buffer Pool Extension (BPE):** Funcionalidad que permite extender la memoria caché de datos (el *Buffer Pool* de la RAM) hacia discos de estado sólido (SSD). Actúa como una caché de nivel 2 (L2), reduciendo la latencia de lectura en entornos donde no hay suficiente RAM, pero sí almacenamiento flash rápido disponible.

SQL Server 2016 (Seguridad, análisis y telemetría)

- **Query Store (Almacén de consultas):** Actúa como una "caja negra" o registrador de vuelo para el motor relacional. Captura automáticamente un historial de consultas, sus planes de ejecución y sus estadísticas de rendimiento (CPU, lecturas, duración) a lo largo del tiempo. Es la herramienta definitiva para detectar regresiones de rendimiento y permite a los DBAs forzar un plan de ejecución óptimo con un solo clic.

  Ver la implementación en C:\Temp\sql\query_store.sql para desplegarlo. Resultado y tests desde SQL Server Management Studio:



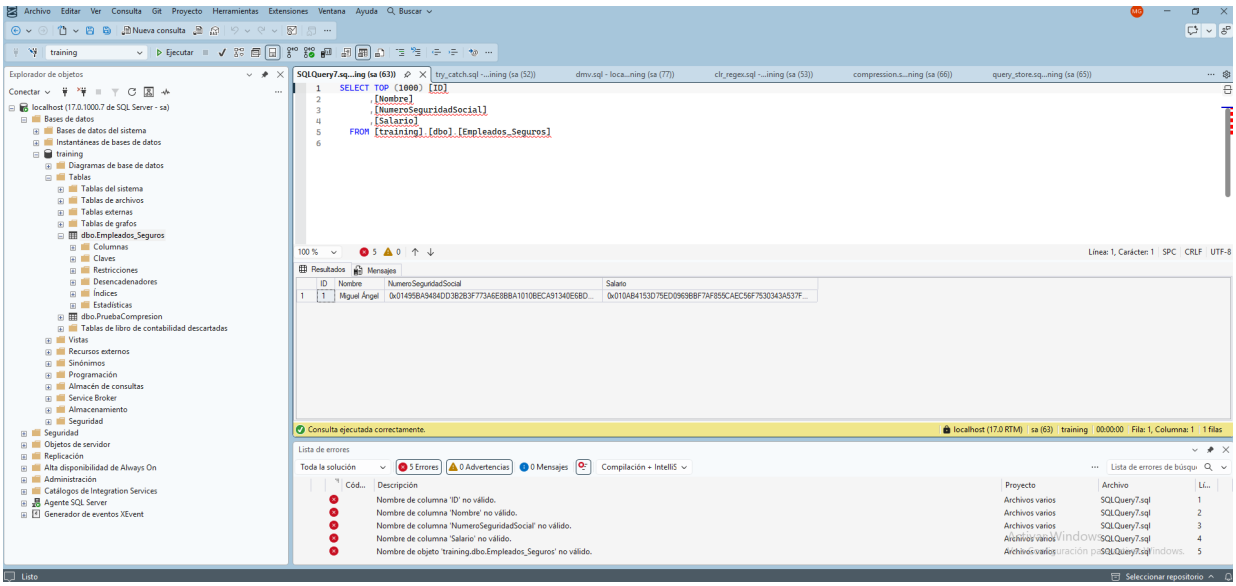
- **Always Encrypted:** Un modelo de seguridad donde los datos se cifran y descifran en el lado del cliente (la aplicación), no en el servidor. Las claves de cifrado nunca se revelan al motor de base de datos. Esto garantiza que ni siquiera los administradores (DBAs o sysadmins) con acceso total al servidor puedan ver los datos confidenciales en texto plano.

  Ver la implementación en C:\git\PruebaAlwaysEncrypted.

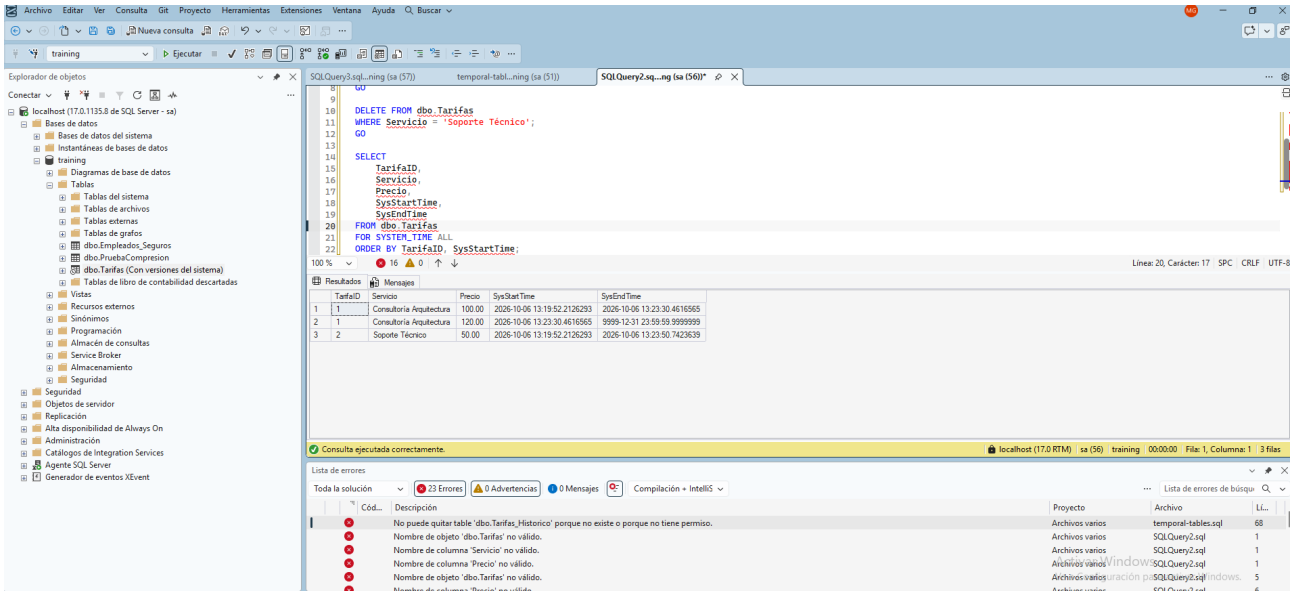
- **Row-Level Security (Seguridad a nivel de fila):** Permite restringir el acceso a los registros de una tabla basándose en el contexto de ejecución del usuario. Mediante políticas de seguridad, se puede lograr que, bajo una misma consulta `SELECT * FROM Ventas`, un vendedor solo vea sus propias ventas, mientras que el gerente regional vea las de todo

el equipo, sin necesidad de modificar el código de la aplicación.

- **Temporal Tables (Tablas temporales con control de versiones):** Tablas que mantienen automáticamente un historial completo de los cambios de datos. El sistema gestiona de forma transparente una tabla histórica adjunta y columnas de periodo (`ValidFrom`, `ValidTo`). Permite



realizar consultas "viajando en el tiempo" usando la cláusula `FOR SYSTEM_TIME AS OF <fecha_hora>`, ideal para auditorías y recuperación de borrados accidentales.



SQL Server 2017 (Multiplataforma e inteligencia)

- **Primera versión en Linux y Docker:** Un hito histórico. Microsoft desacopló el motor de base de datos del sistema operativo Windows mediante una capa de abstracción (SQLPAL). Esto permitió ejecutar SQL Server nativamente en distribuciones como Ubuntu, RHEL y SUSE, y su empaquetado en contenedores Docker, abriendo la puerta a despliegues en Kubernetes y arquitecturas modernas de microservicios.

```
services:
  sqlserver:
      image: mcr.microsoft.com/mssql/server:2022-latest
      container_name: sqlserver-dev
      hostname: sqlserver
      environment:
            - ACCEPT_EULA=Y
            - MSSQL_SA_PASSWORD=TuClaveSegura123! # Requiere mayúsculas,
minúsculas, números y símbolos
            - MSSQL_PID=Developer # Edición gratuita para desarrollo y pruebas
      ports:
            - "1433:1433"
      volumes:
            - sqlserver_data:/var/opt/mssql
      restart: unless-stopped
volumes:
      sqlserver_data:
            name: sqlserver_data
```

- **Adaptive Query Processing (Procesamiento de consultas adaptable):** (Renombrado y ampliado en versiones posteriores como *Intelligent Query Processing*). Hasta entonces, el optimizador generaba un plan de ejecución y lo seguía a ciegas. Con esta mejora, el motor aprende de los errores de estimación en tiempo de ejecución. Por ejemplo, si estima que necesita 10 MB de RAM y usa 1 GB, ajustará automáticamente la concesión de memoria (*Memory Grant Feedback*) para las siguientes ejecuciones de la misma consulta.

SQL Server 2019 (Soporte de caracteres)

- **UTF-8:** Soporte nativo de codificación UTF-8 a nivel de intercalación (*collation*). Permite almacenar caracteres Unicode directamente en columnas estándar `VARCHAR` o `CHAR` (en lugar de requerir el tipo `NVARCHAR`). Para bases de datos con texto mayoritariamente en alfabeto latino que solo necesitan Unicode ocasionalmente, esto reduce el espacio de almacenamiento casi a la mitad.

  UTF-8 (Unicode Transformation Format - 8-bit) es un sistema de codificación de caracteres de longitud variable que traduce el estándar universal Unicode a secuencias de bytes comprensibles para los sistemas informáticos.

  VARCHAR consume 1 byte por carácter (solo puede almacenar un idioma, definido por intercalación (*collation*) de la columna y NVARCHAR consume 2 bytes por carcacter, pero puede almacenar cualquier idioma, sin depender de la configuración regional del sistema operativo o de la base de datos

  Si instalas SQL Server en un entorno en español, el valor por defecto suele ser `Modern_Spanish_CI_AS`. Desglosado, significa que el motor ordenará las letras usando el alfabeto español moderno (donde la 'ch' y la 'll' ya no son letras independientes en el `ORDER BY`), no distinguirá mayúsculas (CI), pero sí será estricto con las tildes (AS).

  Impacto en la arquitectura (El caso de Always Encrypted) Cuando creaste la tabla de ejemplo para probar Always Encrypted, la columna `NumeroSeguridadSocial` exigía el collation `Latin1_General_BIN2`

  Ejemplo del uso de collation en T-SQL:
  -- En un entorno CI_AS, esto devuelve 'Admin', 'ADMIN' o 'admin'

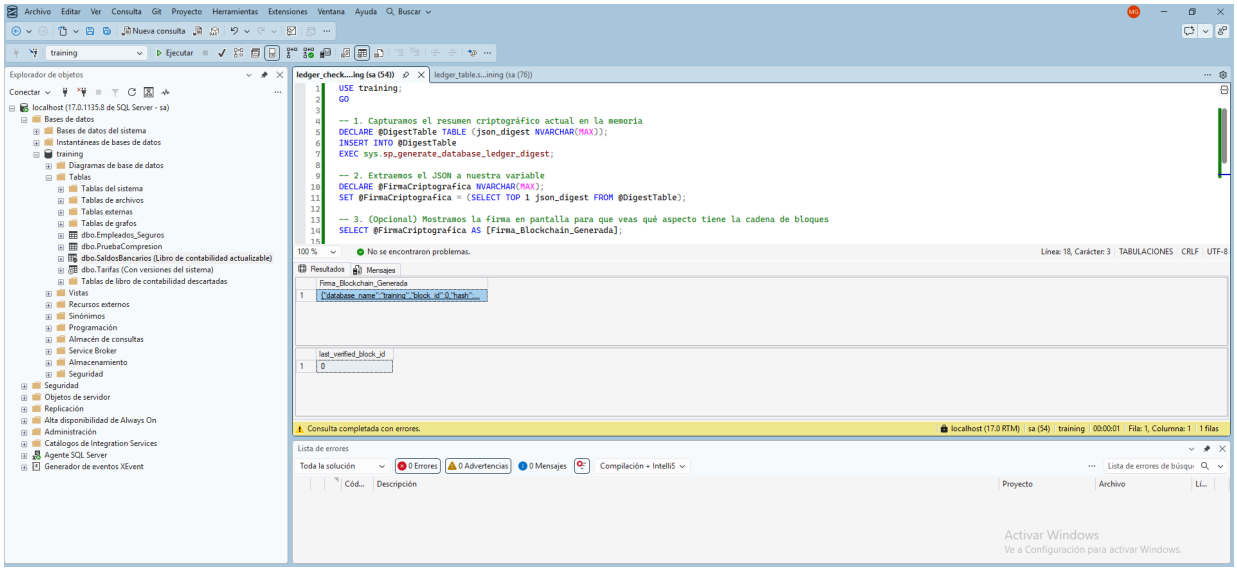
```
SELECT * FROM Usuarios WHERE Username = 'admin';
```

  -- Forzamos la regla para que la base de datos busque la coincidencia exacta (CS)

```
SELECT * FROM Usuarios WHERE Username = 'admin' COLLATE
Latin1_General_CS_AS;
```

SQL Server 2022 (Blockchain, Nube y Optimización Inteligente)

- **Ledger:** Integra conceptos criptográficos de *blockchain* directamente en el motor relacional. Crea tablas a prueba de manipulaciones (*tamper-evident*) mediante la generación de hashes de transacciones. Es ideal para auditorías y cumplimiento normativo, ya que garantiza de forma matemática que nadie (ni siquiera un DBA con privilegios máximos) ha alterado el historial de los datos.



Ver ejemplo en: C:\git\sql-scripts\ledger_table.sql y C:\git\sql-scripts\ledger_check.sql

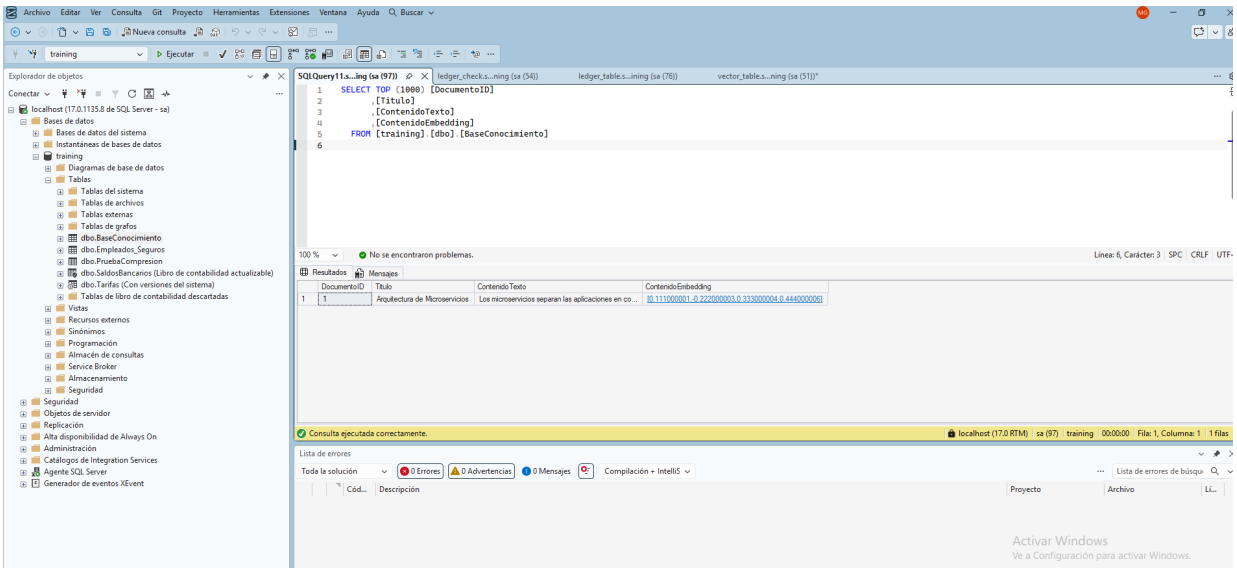
- **Parameter Sensitive Plan (PSP) optimization:** La solución nativa al clásico problema del *Parameter Sniffing*. Anteriormente, si un procedimiento almacenado se compilaba con un parámetro que devolvía 1 fila, el plan se guardaba y se reutilizaba de forma desastrosa para una ejecución posterior que devolviera 10 millones de filas. PSP permite al motor mantener en caché **múltiples planes de ejecución activos** para una misma consulta y usar el más eficiente según la cardinalidad del parámetro actual.

- **Contained Availability Groups (Grupos de disponibilidad contenidos):** Resuelve el mayor punto de fricción operativo de las arquitecturas Always On. En lugar de replicar solo las bases de datos de usuario, un AG contenido incluye sus propias bases de datos de sistema (`master` y `msdb`). Esto significa que los inicios de sesión (*logins*), los permisos y los trabajos del SQL Agent se replican automáticamente entre los nodos del clúster, eliminando la necesidad de sincronizarlos manualmente.

- **Copia a S3 (Backup to URL for S3):** El comando nativo de copias de seguridad amplía su alcance más allá de Azure. Ahora permite ejecutar `BACKUP DATABASE` y `RESTORE` directamente contra cualquier almacenamiento de objetos compatible con la API de Amazon S3, ya sea en la nube pública (AWS, GCP) o en infraestructuras locales (como MinIO).

SQL Server 2025 (IA y Datos Semiestructurados)

- **Tipo vector y funciones de IA:** Transforma el motor transaccional en una base de datos vectorial. Introduce el tipo de dato nativo `VECTOR` para almacenar *embeddings* y funciones integradas para el cálculo de distancias (como la similitud del coseno). Esto permite ejecutar búsquedas semánticas e implementar arquitecturas RAG (Retrieval-Augmented Generation) directamente donde residen los datos, sin depender de bases de datos vectoriales externas.



Ver ejemplo en: C:\git\sql-scripts\vector_table.sql

- **Tipo JSON nativo:** Desde 2016, SQL Server procesaba JSON, pero se almacenaba como texto plano en columnas `NVARCHAR`. El nuevo tipo nativo `JSON` guarda los documentos en un formato binario optimizado para el motor, lo que elimina la sobrecarga de análisis (*parsing*) en tiempo de ejecución y acelera drásticamente las consultas sobre estructuras jerárquicas y esquemas flexibles.

Diapositiva 07-08:
Arquitectura de la DB y clico de vida de una consulta:

Sí, es absolutamente correcto y, de hecho, es el término técnicamente más preciso
T-SQL significa Transact-SQL y es la extensión propietaria de Microsoft del estándar universal ANSI SQL.

Mientras que el "SQL estándar" es un lenguaje puramente declarativo diseñado para interactuar con los datos (hacer SELECT, INSERT, UPDATE, DELETE), T-SQL le añade todas las características de un lenguaje de programación procedural y funcional.

Cuando escribes en SQL Server, casi siempre estás usando T-SQL. Estas son las capacidades clave que T-SQL añade al estándar:

- **Control de flujo:** Permite usar bloques lógicos como IF... ELSE, bucles WHILE, BREAK y CONTINUE.
- **Variables locales:** Te permite declarar y usar variables en la memoria temporal durante la ejecución, como los parámetros que empiezan por arroba (DECLARE @FirmaCriptografica).
- **Manejo de excepciones:** Introduce la captura estructurada de errores mediante bloques BEGIN TRY ... END TRY y BEGIN CATCH ... END CATCH.
- **Procesamiento por lotes y transacciones avanzadas:** Comandos como GO (que separa los lotes de ejecución en herramientas de Microsoft) y la gestión profunda de BEGIN TRAN, COMMIT y ROLLBACK.
- **Funciones propietarias:** Cosas específicas del ecosistema de Microsoft, como las funciones de sistema que acabas de usar (sys.sp_generate_database_ledger_digest) o los tipos de datos nativos exclusivos del motor.

De hecho, en los laboratorios que acabas de realizar, cuando escribiste IF OBJECT_ID('dbo.BaseConocimiento', 'U') IS NOT NULL BEGIN... END para limpiar las tablas de forma segura de forma condicional, estabas programando estrictamente en T-SQL, ya que el SQL puro no soporta esa lógica condicional de esa manera.

¿En qué capa se decide si una consulta usa un Index Seek o un Index Scan? ¿Y quién lo ejecuta?

La decisión la toma el **Optimizador de Consultas (Query Optimizer)** en la capa lógica del Motor Relacional, y la ejecución física la lleva a cabo el **Motor de Almacenamiento (Storage Engine)**.

Esta separación de responsabilidades es la base de la arquitectura cliente-servidor interna de SQL Server:

1. La Decisión: El Optimizador de Consultas (Motor Relacional)

Cuando envías una consulta T-SQL, el motor no la ejecuta inmediatamente. Primero pasa por un analizador sintáctico y luego llega al Optimizador de Consultas. Este componente es el "cerebro" matemático del motor y funciona basándose en costes (Cost-Based Optimizer).

Su trabajo no es encontrar el plan absolutamente perfecto, sino encontrar el plan "suficientemente bueno" en el menor tiempo posible. Para decidir entre un *Seek* o un *Scan*, analiza las **Estadísticas** de la base de datos (histogramas que le dicen cómo están distribuidos los datos en la tabla) y evalúa el "Punto de Inflexión" (*Tipping Point*):

- **Index Seek:** Si las estadísticas le indican que tu cláusula `WHERE` es muy selectiva (por ejemplo, vas a recuperar solo el 1% o menos de las filas), el optimizador decidirá navegar por la estructura de árbol B (B-Tree) del índice saltando directamente a la página de datos que contiene tu registro.
- **Index Scan:** Si le pides demasiados datos (ej. el 30% de la tabla), el optimizador sabe que hacer miles de búsquedas puntuales (Seeks) a través de la estructura del árbol será más lento que simplemente leer todas las páginas de disco secuencialmente desde el principio hasta el final. En ese caso, decide hacer un Scan, incluso si existe un índice.

Una vez tomada la decisión, el Optimizador compila un **Plan de Ejecución** (Execution Plan) y lo guarda en la memoria caché para reutilizarlo en el futuro.

2. La Ejecución: Métodos de Acceso (Motor de Almacenamiento)

Una vez que el plan de ejecución está listo, el Motor Relacional le pasa las instrucciones al **Ejecutor de Consultas**, que a su vez se comunica con la capa inferior: el Motor de Almacenamiento (Storage Engine).

Dentro del Motor de Almacenamiento, existe un componente específico llamado **Métodos de Acceso (Access Methods)**. Este componente no sabe nada de T-SQL, ni de lógica condicional; solo sabe cómo abrir archivos, recorrer estructuras B-Tree y pedirle páginas de datos de 8 KB al gestor de memoria (Buffer Pool).

- Si el plan dicta un **Index Seek**, los Métodos de Acceso piden la página raíz del índice, leen los punteros, piden las páginas intermedias correspondientes y caen directamente en la página hoja exacta donde residen tus datos.

- Si el plan dicta un **Index Scan**, los Métodos de Acceso piden la primera página de la estructura y utilizan unos punteros llamados "forward pointers" para leer secuencialmente página tras página hasta terminar.

Impacto en el código: Cuando hay discrepancias de tipos de datos en la aplicación (como el ejemplo de pasar un parámetro Unicode `NVARCHAR` a una columna `VARCHAR` que vimos antes), el Optimizador de Consultas se confunde, no puede usar las estadísticas correctamente y, por seguridad, le ordenará al Motor de Almacenamiento que haga un Index Scan completo, matando el rendimiento de la consulta.

¿Quieres ver cómo comprobar las estadísticas?

Las **estadísticas** son objetos internos que contienen mapas matemáticos (histogramas y vectores de densidad) sobre la distribución de los datos en una columna o índice. Son la principal fuente de información del Optimizador de Consultas para calcular cuántas filas (cardinalidad) va a devolver tu instrucción.

Si las estadísticas están desactualizadas, el motor "piensa" que una tabla tiene 100 filas cuando en realidad tiene 10 millones, eligiendo un plan de ejecución desastroso.

Aquí tienes cómo auditar y actualizar estos mapas en tu entorno de desarrollo.

1. Inspeccionar el "Cerebro" del Optimizador (DBCC SHOW_STATISTICS)

Para ver exactamente qué sabe SQL Server sobre los datos de una tabla, puedes usar el comando `DBCC SHOW_STATISTICS`.

Supongamos que le has creado un índice a la columna `Titular` de tu tabla `SaldosBancarios`:

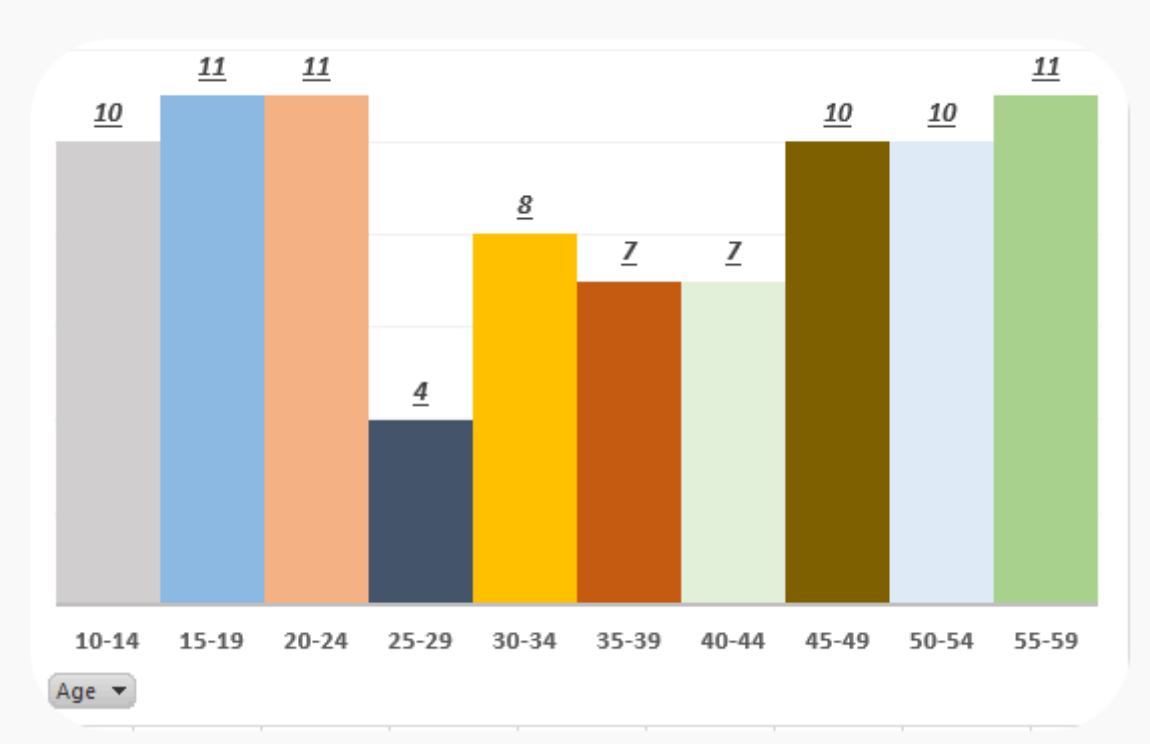
```
USE training;
GO

-- Sintaxis: DBCC SHOW_STATISTICS ('NombreTabla', 'NombreIndice_o_Columna');
DBCC SHOW_STATISTICS ('dbo.SaldosBancarios', 'Titular');
GO
```

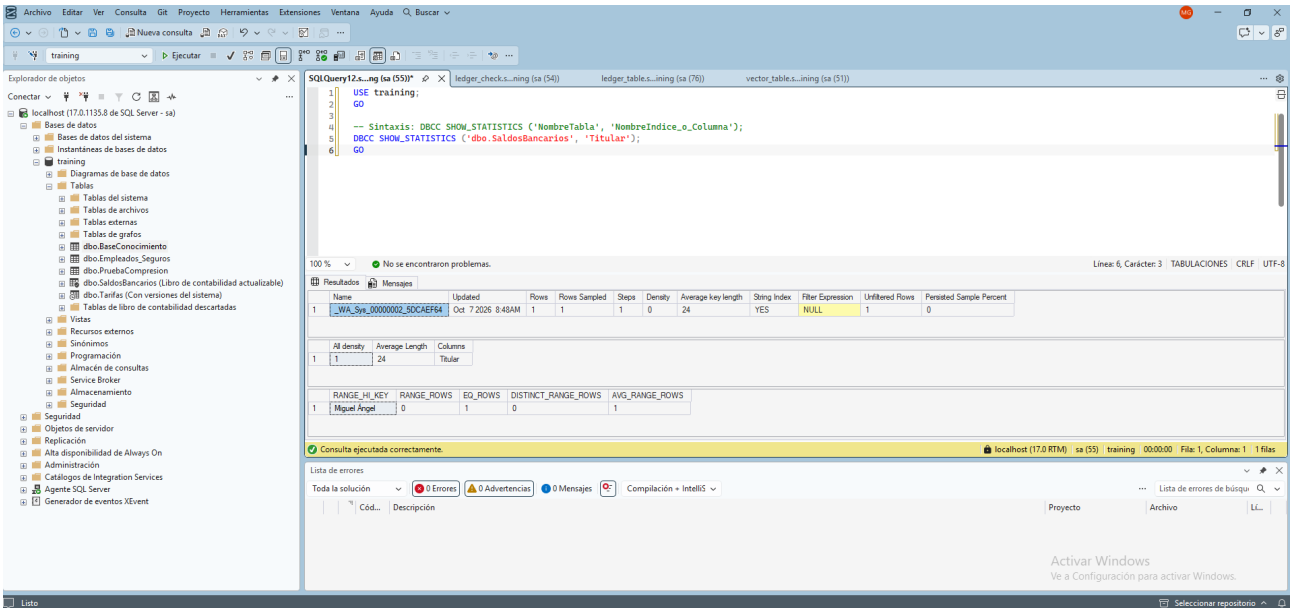
Al ejecutarlo, SQL Server te devolverá tres tablas de resultados que muestran sus cálculos internos:

1. **Encabezado (Header):** Cuándo se actualizaron por última vez y cuántas filas se usaron para calcular el muestreo.

2. **Vector de Densidad (Density Vector):** Una fórmula matemática que mide qué tan únicos son los valores (para saber si vale la pena usar el índice).
3. **Histograma (Histogram):** El desglose físico de los datos.



El límite de los 200 pasos: Sin importar si tu tabla tiene mil filas o mil millones, SQL Server comprimirá la distribución de los datos en un máximo de **200 pasos (steps)** en el histograma. Cada paso indica cuántos registros exactos coinciden con un valor límite (`EQ_ROWS`) y cuántos hay entre un paso y el siguiente (`RANGE_ROWS`).



2. Actualizar las Estadísticas (UPDATE STATISTICS)

Por defecto, SQL Server actualiza las estadísticas automáticamente cuando modificas aproximadamente el 20% de los datos de una tabla. Sin embargo, en tablas enormes (o después de una carga masiva de datos como un BULK INSERT), es vital forzar la actualización para limpiar el caché del optimizador.

Actualización estándar (Muestreo rápido): Lee solo una fracción aleatoria de las páginas de datos para recalcular el mapa matemátio rápidamente sin bloquear el servidor.

```
UPDATE STATISTICS dbo.SaldosBancarios;
GO
```

Actualización exhaustiva (Full Scan): Obliga al motor a leer literalmente todas las filas para generar un mapa matemático perfecto. Es costoso en tiempo de CPU e I/O de disco, pero vital para consultas críticas que están rindiendo mal.

```
UPDATE STATISTICS dbo.SaldosBancarios WITH FULLSCAN; GO
```